Lab 2 – Containerization: Inspect, Understand, Optimize

Babak Huseynov

June 13, 2026

Overview

Picking up the running QuickTicket stack from Lab 1, this lab inspects how the containers are actually built, networked, and resolved – and then optimizes the Dockerfiles by adding `.dockerignore` files and dropping privileges with a non-root user. The bonus traces a single ticket purchase end-to-end through all three application services via timestamped logs.

1 Task 1 – Docker Inspection & Operations

1.1 Image sizes (`docker images | grep app`)

```
REPOSITORY     TAG      IMAGE ID       SIZE
app-gateway    latest   5381e3fa5cb9   239MB
app-events     latest   599e913cda77   260MB
app-payments   latest   688a26ca29ab   237MB
```

The `events` image is the heaviest because its `requirements.txt` pulls in `psycopg2-binary` and the `redis` client on top of FastAPI/uvicorn; the other two only carry the FastAPI stack plus `httpx` (gateway) or nothing extra (payments).

1.2 Layer history (`docker history app-gateway`)

```
CREATED BY                                      SIZE
/bin/sh -c #(nop)  LABEL com.docker.compose...   0B
/bin/sh -c #(nop)  CMD ["uvicorn" "main:app"...  0B
/bin/sh -c #(nop)  EXPOSE 8080                   0B
/bin/sh -c #(nop) COPY file:.../requirements...  24.6kB
/bin/sh -c pip install --no-cache-dir -r req...  28.7MB   <-- pip install
/bin/sh -c #(nop) COPY file:.../main.py          12.3kB
/bin/sh -c #(nop) WORKDIR /app                   8.19kB
CMD ["python3"]                                  0B
RUN /bin/sh -c set -eux;  for src in idle3...    16.4kB
RUN /bin/sh -c set -eux;  savedAptMark=...       43.5MB   <-- Python 3.13 install
... ENV PYTHON_SHA256=..., PYTHON_VERSION=3.13.14, GPG_KEY=...
RUN /bin/sh -c set -eux;  apt-get update;...     4.99MB
ENV PATH=...                                     0B
# debian.sh --arch 'arm64' out/ 'trixie'...      109MB    <-- Debian rootfs
```

Layers and largest layer. The gateway image has 16 history entries (a mix of the inherited `python:3.13-slim` base and our four Dockerfile instructions). The largest single layer is the **109 MB Debian "trixie" rootfs** from the upstream base image. Of our own instructions, the dominant one is exactly the one called out in the lab: `RUN pip install -no-cache-dir -r requirements.txt` at **28.7 MB** (44.1 MB for events, 26.8 MB for payments). That layer is where FastAPI, Starlette, Pydantic, httpx, uvicorn, and prometheus-client land for the gateway. The two COPY layers and `WORKDIR` are all tens of KB or less.

1.3 Container IPs and payments environment

```
$ docker inspect app-events-1   --format '{{.Name}} {{range .NetworkSettings.Networks}}{{.
    IPAddress}}{{end}}'
/app-events-1   172.18.0.5
/app-gateway-1  172.18.0.6
/app-payments-1 172.18.0.4
/app-postgres-1 172.18.0.2
/app-redis-1    172.18.0.3

$ docker inspect app-payments-1 --format '{{range .Config.Env}}{{println .}}{{end}}'
PAYMENT_FAILURE_RATE=0.0
PAYMENT_LATENCY_MS=0
PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
GPG_KEY=7169605F62C751356D054A26A821E680E5FA6305
PYTHON_VERSION=3.13.14
PYTHON_SHA256=639e43243c620a308f968213df9e00f2f8f62332f7adbaa7a7eeb9783057c690
```

1.4 Live debugging with `docker exec`

```
$ docker exec app-gateway-1 whoami
root
$ docker exec app-gateway-1 id
uid=0(root) gid=0(root) groups=0(root)

$ docker exec app-gateway-1 cat /etc/resolv.conf
# Generated by Docker Engine.
nameserver 127.0.0.11
options ndots:0
# ExtServers: [host(192.168.5.1)]

$ docker exec app-gateway-1 python3 -c \
    "import urllib.request, socket; \
     print('events ->', socket.gethostbyname('events')); \
     print(urllib.request.urlopen('http://events:8081/health').read().decode())"
events -> 172.18.0.5
{"status":"healthy","checks":{"postgres":"ok","redis":"ok"}}

$ docker exec app-gateway-1 python3 -c \
    "import urllib.request, socket; \
     print('payments ->', socket.gethostbyname('payments')); \
     print(urllib.request.urlopen('http://payments:8082/health').read().decode())"
payments -> 172.18.0.4
{"status":"healthy","failure_rate":0.0,"latency_ms":0}
```

1.5 Logs: matching one request across gateway and events

After hitting `POST /events/1/reserve` once, the gateway and events tails show the same event picked up on both sides within 1 ms:

```
# gateway
{"time":"2026-06-12 21:14:37,438","service":"gateway",
 "msg":"HTTP Request: POST http://events:8081/events/1/reserve "HTTP/1.1 200 OK""}
INFO: 172.18.0.1:36534 - "POST /events/1/reserve HTTP/1.1" 200 OK

# events
{"time":"2026-06-12 21:14:37,437","service":"events",
 "msg":"Reserved 1 tickets for event 1: 66c38061-b896-4d74-8e0f-7b5d8f8e551d"}
INFO: 172.18.0.6:58192 - "POST /events/1/reserve HTTP/1.1" 200 OK
```

The source IP `172.18.0.6` in the events log is the gateway container, which confirms the call did not skip the gateway and that timestamps line up to the millisecond.

1.6 Network inspect

```
$ docker network ls | grep app
86d313a1749a   app_default   bridge    local

$ docker network inspect app_default \
    --format '{{range .Containers}}{{.Name}}: {{.IPv4Address}}{{println ""}}{{end}}'
app-postgres-1: 172.18.0.2/16
app-gateway-1:  172.18.0.6/16
app-events-1:   172.18.0.5/16
app-redis-1:    172.18.0.3/16
app-payments-1: 172.18.0.4/16
```

How does the gateway find `events`? What IP does it resolve to?

Docker Compose put every container in this stack onto a single user-defined bridge network called `app_default`. On user-defined bridges, Docker runs an embedded DNS server inside each container, reachable at `127.0.0.11` – you can see it in `/etc/resolv.conf` above. That resolver knows the Compose service names (`gateway`, `events`, `payments`, `postgres`, `redis`) and maps each one to the current container's IP on the bridge subnet `172.18.0.0/16`. So when the gateway code does `httpx.AsyncClient.get("http://events:8081/...")`, glibc asks `127.0.0.11`, gets back `172.18.0.5`, and the kernel routes that across the bridge. There is no hardcoded IP and no `/etc/hosts` entry – if the events container is recreated and lands on a different IP, the next DNS lookup transparently picks up the new address.

2 Task 2 – Dockerfile Optimization

2.1 `.dockerignore`

The same file was added to each of `app/gateway/`, `app/events/`, and `app/payments/`:

```
__pycache__
*.pyc
.git
.env
*.md
.vscode
```

2.2 Image sizes before and after

Image	Before	After (`-no-cache` rebuild + .dockerignore + USER app)
`app-gateway`	239 MB	239 MB
`app-events`	260 MB	260 MB
`app-payments`	237 MB	237 MB

No measurable change, which is the expected result. The hint in the lab calls this out: `.dockerignore` only saves space if the build context actually contains the ignored files, and these three service directories ship a single `main.py` plus `requirements.txt` – there is no `.git/`, no `__pycache__/`, no `*.md` sitting next to the Dockerfile. The win here is preventive rather than immediate: once a developer drops a `.env` file or runs the app locally and generates `__pycache__/`, the next build will not silently pull those into the image (or worse, leak secrets through layer history).

2.3 Non-root user (after rebuild)

```
$ docker exec app-gateway-1 whoami
app
$ docker exec app-gateway-1 id
uid=100(app) gid=101(app) groups=101(app)

$ docker exec app-events-1 whoami
app
$ docker exec app-payments-1 whoami
app
```

All three application containers now run as the unprivileged `app` user instead of `root`. `postgres` and `redis` are unchanged because they use upstream images that already manage their own users.

2.4 Dockerfile diff

The same three-line block was added to each of `gateway/`, `events/`, and `payments/` Dockerfiles (representative diff for gateway):

```
diff --git a/app/gateway/Dockerfile b/app/gateway/Dockerfile
@@
 COPY main.py .

+RUN addgroup --system app && adduser --system --ingroup app app
+USER app
+
 EXPOSE 8080
 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
```

The `USER app` line is placed *after* the `COPY` of `main.py` so the file is owned by root (which is fine – it is only read at startup) but the process running uvicorn drops privileges. No `chown` was required because the app does not write to disk; if it did, the `/app` directory would need `RUN chown -R app:app /app` before `USER app`, as the lab hint warns.

3 Bonus Task – Trace a Request Across Services

The stack was recycled with `docker compose down && docker compose up -d`, the system left to idle for ~15 s, then a single full purchase was executed. The captured response was:

```
reservation_id = 609d81dc-2525-4b36-b456-043f50c1edfb
order_id       = 609d81dc-2525-4b36-b456-043f50c1edfb    (status: confirmed)
```

Timestamped logs filtered to this reservation ID (sorted by service for readability):

```
# gateway
21:18:37.072152  POST http://events:8081/events/1/reserve -> 200 OK
21:18:37.072631  "POST /events/1/reserve HTTP/1.1" 200 OK           <- response to client
21:18:37.106771  POST http://payments:8082/charge -> 200 OK
21:18:37.109419  POST http://events:8081/reservations/609d.../confirm -> 200 OK
21:18:37.109977  "POST /reserve/609d.../pay HTTP/1.1" 200 OK        <- response to client

# events
21:18:37.071553  Reserved 1 tickets for event 1: 609d81dc-...       <- DB read + Redis SETEX
21:18:37.071864  "POST /events/1/reserve HTTP/1.1" 200 OK
21:18:37.108971  Order confirmed: 609d81dc-...                      <- INSERT INTO orders
21:18:37.109165  "POST /reservations/609d.../confirm HTTP/1.1" 200 OK
```

```
# payments
21:18:37.106256  Payment success: PAY-835E7A9E for 609d81dc-...      <- mock charge
21:18:37.106462  "POST /charge HTTP/1.1" 200 OK
```

3.1 Annotated walk-through

#	Time (s)	Hop	What happened
1	37.07155	events	`POST /events/1/reserve`: looks up event 1 in Postgres, `SETEX reservation:...` and `DECRBY event:1:held` in Redis.
2	37.07186	events	Response sent to gateway.
3	37.07215	gateway	httpx records the downstream call returned 200.
4	37.07263	gateway	Client receives the `reservation_id` (end of reserve flow).
5	37.10626	payments	`POST /charge`: mock processor generates `PAY-835E7A9E`.
6	37.10646	payments	Response sent to gateway.
7	37.10677	gateway	httpx records charge returned 200.
8	37.10897	events	`POST /reservations/.../confirm`: `INSERT INTO orders`, commits, `DEL reservation:...` in Redis.
9	37.10916	events	Confirm response sent to gateway.
10	37.10942	gateway	httpx records confirm returned 200.
11	37.10998	gateway	Client receives `order_id` (end of pay flow).

3.2 End-to-end timing

Reserve flow (steps 1–4): events finished at 37.07155 s, gateway responded at 37.07263 s. End-to-end **≈1.1 ms** from when events started handling the reserve to when the client got its reservation. The work is dominated by one Postgres SELECT, one Redis SETEX, and one Redis DECRBY.

Pay flow (steps 5–11): the first downstream call (charge) registered on the payments side at 37.10626 s and the gateway returned to the client at 37.10998 s. End-to-end **≈3.7 ms** for the entire pay handler, which contains *two* sequential service hops (gateway→payments, then gateway→events confirm) and an INSERT to Postgres. Per hop the breakdown is roughly: charge round-trip ˜0.5 ms, confirm round-trip ˜2.6 ms (dominated by Postgres write + commit), and ˜0.5 ms of gateway glue.

Total client-perceived time for a complete purchase (reserve + pay, as a sequential choreography from the client's point of view) is therefore on the order of **5 ms**, matching what `docker stats` showed at 10 RPS: the system was nowhere near its CPU budget.